

Serverless Event-Driven Data Pipeline

Real-Time Data Processing, Transformation & Global Delivery

- AWS Lambda
- Amazon S3
- Amazon DynamoDB
- Amazon CloudFront
- Python & Boto3
- Amazon CloudWatch

1. PROJECT OVERVIEW & ARCHITECTURE OBJECTIVES

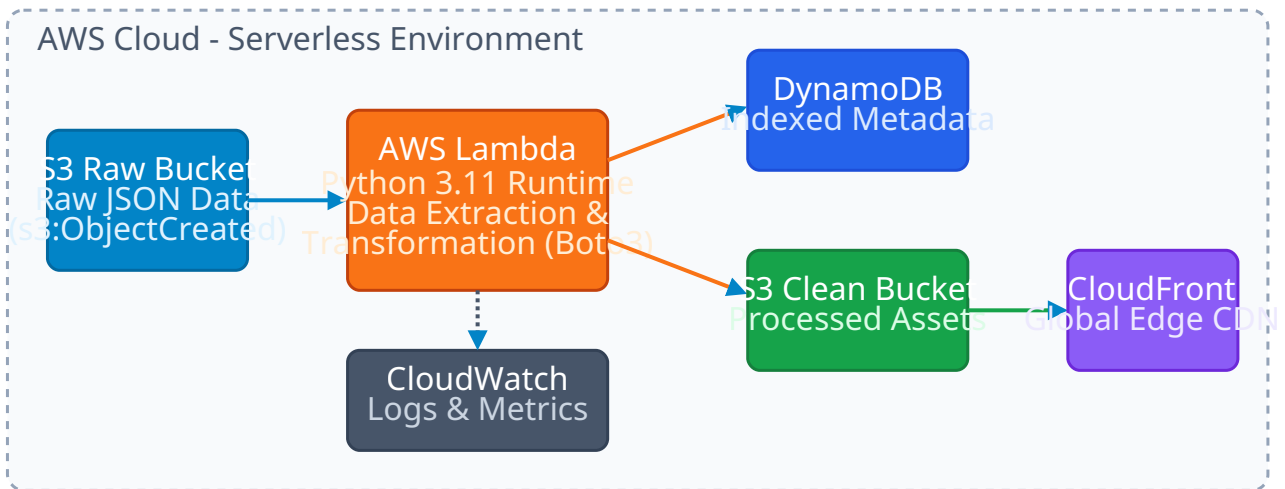
Traditional batch data processing pipelines suffer from latency, server maintenance overhead, and idle compute costs. This project details the design and deployment of a **Serverless Event-Driven Data Processing Pipeline** on AWS.

Whenever unstructured JSON data payloads are ingested into an Amazon S3 Raw Bucket, S3 event notifications automatically trigger a serverless Python AWS Lambda function. The function parses, validates, enriches, and transforms the incoming data in real time using Boto3, indexing structured records into Amazon DynamoDB and archiving processed files into a Clean S3 Bucket. The processed assets are served globally with low latency via Amazon CloudFront.

Architecture Layer	AWS Service	Technical Function & Purpose
Ingestion & Triggers	Amazon S3 (Raw Bucket)	Stores raw JSON files and fires <code>s3:ObjectCreated:*</code> event notifications.
Compute Engine	AWS Lambda (Python 3.11)	Serverless execution environment handling ETL, validation, and DynamoDB writes.
Data Persistence	Amazon DynamoDB	NoSQL database storing indexed key-value records with Partition (PK) & Sort keys (SK).
Content Delivery	Amazon CloudFront	Edge-cached CDN delivering clean archived assets globally with Origin Access Control.
Observability	Amazon CloudWatch	Centralized logging, error metric tracking, and alarm triggers for pipeline failures.

2. SYSTEM ARCHITECTURE & EVENT FLOW DIAGRAM

FIGURE 1: SERVERLESS EVENT-DRIVEN ARCHITECTURE PIPELINE



3. CORE PIPELINE EXECUTION STEPS

3.1 Event Triggers & Ingestion

The ingestion layer relies on asynchronous S3 bucket event notifications. Upon uploading a file ending in .json to s3://raw-data-bucket/, S3 sends an event payload containing bucket details, object keys, and file timestamps directly to the Lambda service.

3.2 Serverless Transformation (AWS Lambda & Boto3)

The Lambda function executes in an isolated environment with access to standard Python libraries and the AWS SDK (Boto3). The handler performs the following steps:

- Reads raw file stream from Amazon S3 into memory.
- Parses and validates schema integrity (rejecting corrupt payloads).
- Enriches record metadata by injecting processing timestamps and UUIDs.
- Writes structured attributes into DynamoDB and archives transformed files into the Clean S3 bucket.

3.3 Low-Latency Distribution (CloudFront & OAC)

Processed output files stored in the Clean S3 bucket are made available globally via Amazon CloudFront. Access to the S3 origin is strictly restricted using **Origin Access Control (OAC)**, ensuring files cannot be read directly from S3 without passing through CloudFront's security layer.

4. KEY PYTHON HANDLER LOGIC SNIPPET

```
import json
import urllib.parse
import boto3
import uuid
from datetime import datetime

s3_client = boto3.client('s3')
dynamodb = boto3.resource('dynamodb')

TABLE_NAME = "ServerlessProcessedData"
CLEAN_BUCKET = "my-clean-data-bucket"

def lambda_handler(event, context):
    try:
        # 1. Parse Event Details
        record = event['Records'][0]
        src_bucket = record['s3']['bucket']['name']
        src_key = urllib.parse.unquote_plus(record['s3']['object']['key'], encoding='utf-8')

        # 2. Retrieve Raw Object from S3
        response = s3_client.get_object(Bucket=src_bucket, Key=src_key)
        raw_content = response['Body'].read().decode('utf-8')
        data = json.loads(raw_content)

        # 3. Transform & Enrich
        data['record_id'] = str(uuid.uuid4())
        data['processed_at'] = datetime.utcnow().isoformat()
        data['source_file'] = src_key

        # 4. Save to DynamoDB
        table = dynamodb.Table(TABLE_NAME)
        table.put_item(Item=data)

        # 5. Archive Clean File
        s3_client.put_object(
            Bucket=CLEAN_BUCKET,
            Key=f"processed/{data['record_id']}.json",
            Body=json.dumps(data),
            ContentType='application/json'
        )

        return {'statusCode': 200, 'body': 'Pipeline Execution Successful'}

    except Exception as e:
        print(f"Error processing pipeline: {str(e)}")
        raise e
```

5. KEY ARCHITECTURAL ACHIEVEMENTS

Pipeline Benefits & Performance

- **Zero Server Administration:** Fully serverless operational model eliminates OS patching and capacity planning.
- **Sub-Second Processing Latency:** Automated event invocation processes incoming JSON objects in milliseconds.
- **Pay-per-Use Cost Model:** Incurs zero cost when inactive, auto-scaling instantly during heavy data ingestion spikes.

- **End-to-End Monitoring:** CloudWatch integration logs execution metrics and alerts on validation or runtime errors.